JavaScript is disabled on your browser.



compact1, compact2, compact3

java.lang

# Class Runtime

java.lang.Object
    java.lang.Runtime

---

```
public class Runtime
extends Object
```

Every Java application has a single instance of class `Runtime` that allows the application to interface with the environment in which the application is running. The current runtime can be obtained from the `getRuntime` method.

An application cannot create its own instance of this class.

**Since:**

`JDK1.0`

**See Also:**

`getRuntime()`

## *Method Summary*

Concrete Methods | Deprecated Methods

| Modifier and Type | Method | Description |
|---|---|---|
| `void` | `addShutdownHook(Thread hook)` | Registers a new virtual-machine shutdown hook. |
| `int` | `availableProcessors()` | Returns the number of processors available to |

| | | |
|---|---|---|
| | | the Java virtual machine. |
| **Process** | **exec**(**String** command) | Executes the specified string command in a separate process. |
| **Process** | **exec**(**String**[] cmdarray) | Executes the specified command and arguments in a separate process. |
| **Process** | **exec**(**String**[] cmdarray, **String**[] envp) | Executes the specified command and arguments in a separate process with the specified environment. |
| **Process** | **exec**(**String**[] cmdarray, **String**[] envp, **File** dir) | Executes the specified command and arguments in a separate process with the specified environment and working directory. |
| **Process** | **exec**(**String** command, **String**[] envp) | Executes the specified string command in a separate process with the specified environment. |
| **Process** | **exec**(**String** command, **String**[] envp, **File** dir) | Executes the specified string command in a separate process with the specified environment and working directory. |
| void | **exit**(int status) | Terminates the currently running Java virtual machine by initiating its shutdown sequence. |
| long | **freeMemory**() | Returns the amount of free memory in the Java Virtual Machine. |

| | | |
|---|---|---|
| void | **gc**() | Runs the garbage collector. |
| **InputStream** | **getLocalizedInputStream**(**InputStream** in) | **Deprecated.** As of JDK 1.1, the preferred way to translate a byte stream in the local encoding into a character stream in Unicode is via the `InputStreamReader` and `BufferedReader` classes. |
| **OutputStream** | **getLocalizedOutputStream**(**OutputStream** out) | **Deprecated.** As of JDK 1.1, the preferred way to translate a Unicode character stream into a byte stream in the local encoding is via the `OutputStreamWriter`, `BufferedWriter`, and `PrintWriter` classes. |
| static **Runtime** | **getRuntime**() | Returns the runtime object associated with the current Java application. |
| void | **halt**(int status) | Forcibly terminates the currently running Java virtual machine. |
| void | **load**(**String** filename) | Loads the native library specified by the filename argument. |
| void | **loadLibrary**(**String** libname) | Loads the native library specified by the `libname` argument. |
| long | **maxMemory**() | Returns the maximum amount of memory that the Java virtual |

| | | |
|---|---|---|
| | | machine will attempt to use. |
| boolean | **removeShutdownHook**(**Thread** hook) | De-registers a previously-registered virtual-machine shutdown hook. |
| void | **runFinalization**() | Runs the finalization methods of any objects pending finalization. |
| static void | **runFinalizersOnExit**(boolean value) | **Deprecated.** This method was originally designed to enable or disable running finalizers on exit. Running finalizers on exit was disabled by default. If enabled, then the finalizers of all objects whose finalizers had not yet been automatically invoked were to be run before the Java runtime exits. That behavior is inherently unsafe. It may result in finalizers being called on live objects while other threads are concurrently manipulating those objects, resulting in erratic behavior or deadlock. |
| long | **totalMemory**() | Returns the total amount of memory in the Java virtual machine. |
| void | **traceInstructions**(boolean on) | Enables/Disables tracing of instructions. |
| void | **traceMethodCalls**(boolean on) | |

| | | |
|---|---|---|
| | | Enables/Disables tracing of method calls. |

### Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

## *Method Detail*

### getRuntime

```
public static Runtime getRuntime()
```

Returns the runtime object associated with the current Java application. Most of the methods of class `Runtime` are instance methods and must be invoked with respect to the current runtime object.

**Returns:**

the Runtime object associated with the current Java application.

### exit

```
public void exit(int status)
```

Terminates the currently running Java virtual machine by initiating its shutdown sequence. This method never returns normally. The argument serves as a status code; by convention, a nonzero status code indicates abnormal termination.

All registered shutdown hooks, if any, are started in some unspecified order and allowed to run concurrently until they finish. Once this is done the virtual machine halts.

If this method is invoked after all shutdown hooks have already been run and the status is nonzero then this method halts the virtual machine with the given status code. Otherwise, this method blocks indefinitely.

The `System.exit` method is the conventional and convenient means of invoking this method.

**Parameters:**

status - Termination status. By convention, a nonzero status code indicates abnormal termination.

**Throws:**

`SecurityException` - If a security manager is present and its `checkExit` method does not permit exiting with the specified status

**See Also:**

`SecurityException`, `SecurityManager.checkExit(int)`, `addShutdownHook(java.lang.Thread)`, `removeShutdownHook(java.lang.Thread)`, `halt(int)`

### addShutdownHook

```
public void addShutdownHook(Thread hook)
```

Registers a new virtual-machine shutdown hook.

The Java virtual machine *shuts down* in response to two kinds of events:

- The program *exits* normally, when the last non-daemon thread exits or when the `exit` (equivalently, `System.exit`) method is invoked, or
- The virtual machine is *terminated* in response to a user interrupt, such as typing `^C`, or a system-wide event, such as user logoff or system shutdown.

A *shutdown hook* is simply an initialized but unstarted thread. When the virtual machine begins its shutdown sequence it will start all registered shutdown hooks in some unspecified order and let them run concurrently. When all the hooks have finished it will then halt. Note that daemon threads will continue to run during the shutdown sequence, as will non-daemon threads if shutdown was initiated by invoking the `exit` method.

Once the shutdown sequence has begun it can be stopped only by invoking the `halt` method, which forcibly terminates the virtual machine.

Once the shutdown sequence has begun it is impossible to register a new shutdown hook or de-register a previously-registered hook. Attempting either of these operations will cause an `IllegalStateException` to be thrown.

Shutdown hooks run at a delicate time in the life cycle of a virtual machine and should therefore be coded defensively. They should, in particular, be written to be thread-safe and to avoid deadlocks insofar as possible. They should also not rely blindly upon services that may have registered their own shutdown hooks and therefore may themselves in the process of shutting down. Attempts to use other thread-based services such as the AWT event-dispatch thread, for example, may lead to deadlocks.

Shutdown hooks should also finish their work quickly. When a program invokes `exit` the expectation is that the virtual machine will promptly shut down and exit. When the virtual machine is terminated due to user logoff or system shutdown the underlying operating system may only allow a fixed amount of time in which to shut down and exit. It is therefore inadvisable to

attempt any user interaction or to perform a long-running computation in a shutdown hook.

Uncaught exceptions are handled in shutdown hooks just as in any other thread, by invoking the `uncaughtException` method of the thread's `ThreadGroup` object. The default implementation of this method prints the exception's stack trace to `System.err` and terminates the thread; it does not cause the virtual machine to exit or halt.

In rare circumstances the virtual machine may *abort*, that is, stop running without shutting down cleanly. This occurs when the virtual machine is terminated externally, for example with the `SIGKILL` signal on Unix or the `TerminateProcess` call on Microsoft Windows. The virtual machine may also abort if a native method goes awry by, for example, corrupting internal data structures or attempting to access nonexistent memory. If the virtual machine aborts then no guarantee can be made about whether or not any shutdown hooks will be run.

**Parameters:**

`hook - An initialized but unstarted Thread object`

**Throws:**

`IllegalArgumentException - If the specified hook has already been registered, or if it can be determined that the hook is already running or has already been run`

`IllegalStateException - If the virtual machine is already in the process of shutting down`

`SecurityException - If a security manager is present and it denies RuntimePermission("shutdownHooks")`

**Since:**

`1.3`

**See Also:**

`removeShutdownHook(java.lang.Thread), halt(int), exit(int)`

### removeShutdownHook

```
public boolean removeShutdownHook(Thread hook)
```

De-registers a previously-registered virtual-machine shutdown hook.

**Parameters:**

`hook - the hook to remove`

**Returns:**

`true if the specified hook had previously been registered and was successfully de-registered, false otherwise.`

**Throws:**

IllegalStateException - If the virtual machine is already in the process of shutting down

SecurityException - If a security manager is present and it denies RuntimePermission("shutdownHooks")

**Since:**

1.3

**See Also:**

addShutdownHook(java.lang.Thread), exit(int)

### halt

```
public void halt(int status)
```

Forcibly terminates the currently running Java virtual machine. This method never returns normally.

This method should be used with extreme caution. Unlike the exit method, this method does not cause shutdown hooks to be started. If the shutdown sequence has already been initiated then this method does not wait for any running shutdown hooks to finish their work.

**Parameters:**

status - Termination status. By convention, a nonzero status code indicates abnormal termination. If the exit (equivalently, System.exit) method has already been invoked then this status code will override the status code passed to that method.

**Throws:**

SecurityException - If a security manager is present and its checkExit method does not permit an exit with the specified status

**Since:**

1.3

**See Also:**

exit(int), addShutdownHook(java.lang.Thread), removeShutdownHook(java.lang.Thread)

### runFinalizersOnExit

```
@Deprecated
public static void runFinalizersOnExit(boolean value)
```

**Deprecated.** *This method was originally designed to enable or disable running finalizers on exit. Running finalizers on exit was disabled by default. If enabled, then the finalizers of all objects whose finalizers had not yet been automatically invoked were to be run before the Java runtime exits. That*

*behavior is inherently unsafe. It may result in finalizers being called on live objects while other threads are concurrently manipulating those objects, resulting in erratic behavior or deadlock.*

Throws `UnsupportedOperationException`.

**Parameters:**

`value - ignored`

**Since:**

`JDK1.1`

### exec

```
public Process exec(String command)
             throws IOException
```

Executes the specified string command in a separate process.

This is a convenience method. An invocation of the form `exec(command)` behaves in exactly the same way as the invocation `exec(command, null, null)`.

**Parameters:**

`command - a specified system command.`

**Returns:**

`A new Process object for managing the subprocess`

**Throws:**

`SecurityException - If a security manager exists and its checkExec method doesn't allow creation of the subprocess`

`IOException - If an I/O error occurs`

`NullPointerException - If command is null`

`IllegalArgumentException - If command is empty`

**See Also:**

`exec(String[], String[], File), ProcessBuilder`

### exec

```
public Process exec(String command,
                    String[] envp)
             throws IOException
```

Executes the specified string command in a separate process with the specified environment.

This is a convenience method. An invocation of the form `exec(command, envp)` behaves in exactly the same way as the invocation `exec(command, envp, null)`.

**Parameters:**

`command - a specified system command.`

`envp - array of strings, each element of which has environment variable settings in the format name=value, or null if the subprocess should inherit the environment of the current process.`

**Returns:**

`A new Process object for managing the subprocess`

**Throws:**

`SecurityException - If a security manager exists and its checkExec method doesn't allow creation of the subprocess`

`IOException - If an I/O error occurs`

`NullPointerException - If command is null, or one of the elements of envp is null`

`IllegalArgumentException - If command is empty`

**See Also:**

`exec(String[], String[], File), ProcessBuilder`

### exec

```
public Process exec(String command,
                    String[] envp,
                    File dir)
             throws IOException
```

Executes the specified string command in a separate process with the specified environment and working directory.

This is a convenience method. An invocation of the form `exec(command, envp, dir)` behaves in exactly the same way as the invocation `exec(cmdarray, envp, dir)`, where `cmdarray` is an array of all the tokens in `command`.

More precisely, the `command` string is broken into tokens using a `StringTokenizer` created by the call `new StringTokenizer(command)` with no further modification of the character categories. The tokens produced by the tokenizer are then placed in the new string array `cmdarray`, in the same order.

**Parameters:**

`command - a specified system command.`

envp - array of strings, each element of which has environment variable settings in the format *name*=*value*, or null if the subprocess should inherit the environment of the current process.

dir - the working directory of the subprocess, or null if the subprocess should inherit the working directory of the current process.

**Returns:**

A new Process object for managing the subprocess

**Throws:**

SecurityException - If a security manager exists and its checkExec method doesn't allow creation of the subprocess

IOException - If an I/O error occurs

NullPointerException - If command is null, or one of the elements of envp is null

IllegalArgumentException - If command is empty

**Since:**

1.3

**See Also:**

ProcessBuilder

### exec

```
public Process exec(String[] cmdarray)
             throws IOException
```

Executes the specified command and arguments in a separate process.

This is a convenience method. An invocation of the form exec(cmdarray) behaves in exactly the same way as the invocation exec(cmdarray, null, null).

**Parameters:**

cmdarray - array containing the command to call and its arguments.

**Returns:**

A new Process object for managing the subprocess

**Throws:**

SecurityException - If a security manager exists and its checkExec method doesn't allow creation of the subprocess

IOException - If an I/O error occurs

NullPointerException - If cmdarray is null, or one of the elements of cmdarray is null

IndexOutOfBoundsException - If cmdarray is an empty array (has length 0)

**See Also:**

ProcessBuilder

### exec

```
public Process exec(String[] cmdarray,
                    String[] envp)
             throws IOException
```

Executes the specified command and arguments in a separate process with the specified environment.

This is a convenience method. An invocation of the form exec(cmdarray, envp) behaves in exactly the same way as the invocation exec(cmdarray, envp, null).

**Parameters:**

cmdarray - array containing the command to call and its arguments.

envp - array of strings, each element of which has environment variable settings in the format *name*=*value*, or null if the subprocess should inherit the environment of the current process.

**Returns:**

A new Process object for managing the subprocess

**Throws:**

SecurityException - If a security manager exists and its checkExec method doesn't allow creation of the subprocess

IOException - If an I/O error occurs

NullPointerException - If cmdarray is null, or one of the elements of cmdarray is null, or one of the elements of envp is null

IndexOutOfBoundsException - If cmdarray is an empty array (has length 0)

**See Also:**

ProcessBuilder

### exec

```
public Process exec(String[] cmdarray,
                    String[] envp,
                    File dir)
             throws IOException
```

Executes the specified command and arguments in a separate process with the specified environment and working directory.

Given an array of strings `cmdarray`, representing the tokens of a command line, and an array of strings `envp`, representing "environment" variable settings, this method creates a new process in which to execute the specified command.

This method checks that `cmdarray` is a valid operating system command. Which commands are valid is system-dependent, but at the very least the command must be a non-empty list of non-null strings.

If `envp` is `null`, the subprocess inherits the environment settings of the current process.

A minimal set of system dependent environment variables may be required to start a process on some operating systems. As a result, the subprocess may inherit additional environment variable settings beyond those in the specified environment.

`ProcessBuilder.start()` is now the preferred way to start a process with a modified environment.

The working directory of the new subprocess is specified by `dir`. If `dir` is `null`, the subprocess inherits the current working directory of the current process.

If a security manager exists, its `checkExec` method is invoked with the first component of the array `cmdarray` as its argument. This may result in a `SecurityException` being thrown.

Starting an operating system process is highly system-dependent. Among the many things that can go wrong are:

- The operating system program file was not found.
- Access to the program file was denied.
- The working directory does not exist.

In such cases an exception will be thrown. The exact nature of the exception is system-dependent, but it will always be a subclass of `IOException`.

**Parameters:**

`cmdarray - array containing the command to call and its arguments.`

`envp - array of strings, each element of which has environment variable settings in the format` *`name=value`*`, or null if the subprocess should inherit the environment of the current process.`

`dir - the working directory of the subprocess, or null if the subprocess should inherit the working directory of the current process.`

**Returns:**

`A new Process object for managing the subprocess`

**Throws:**

SecurityException - If a security manager exists and its checkExec method doesn't allow creation of the subprocess

IOException - If an I/O error occurs

NullPointerException - If cmdarray is null, or one of the elements of cmdarray is null, or one of the elements of envp is null

IndexOutOfBoundsException - If cmdarray is an empty array (has length 0)

**Since:**

1.3

**See Also:**

ProcessBuilder

### availableProcessors

```
public int availableProcessors()
```

Returns the number of processors available to the Java virtual machine.

This value may change during a particular invocation of the virtual machine. Applications that are sensitive to the number of available processors should therefore occasionally poll this property and adjust their resource usage appropriately.

**Returns:**

the maximum number of processors available to the virtual machine; never smaller than one

**Since:**

1.4

### freeMemory

```
public long freeMemory()
```

Returns the amount of free memory in the Java Virtual Machine. Calling the gc method may result in increasing the value returned by freeMemory.

**Returns:**

an approximation to the total amount of memory currently available for future allocated objects, measured in bytes.

### totalMemory

```
public long totalMemory()
```

Returns the total amount of memory in the Java virtual machine. The value returned by this method may vary over time, depending on the host environment.

Note that the amount of memory required to hold an object of any given type may be implementation-dependent.

**Returns:**

the total amount of memory currently available for current and future objects, measured in bytes.

### maxMemory

```
public long maxMemory()
```

Returns the maximum amount of memory that the Java virtual machine will attempt to use. If there is no inherent limit then the value `Long.MAX_VALUE` will be returned.

**Returns:**

the maximum amount of memory that the virtual machine will attempt to use, measured in bytes

**Since:**

1.4

### gc

```
public void gc()
```

Runs the garbage collector. Calling this method suggests that the Java virtual machine expend effort toward recycling unused objects in order to make the memory they currently occupy available for quick reuse. When control returns from the method call, the virtual machine has made its best effort to recycle all discarded objects.

The name `gc` stands for "garbage collector". The virtual machine performs this recycling process automatically as needed, in a separate thread, even if the `gc` method is not invoked explicitly.

The method `System.gc()` is the conventional and convenient means of invoking this method.

### runFinalization

```
public void runFinalization()
```

Runs the finalization methods of any objects pending finalization. Calling this method suggests that the Java virtual machine expend effort toward running the `finalize` methods of objects that have been found to be discarded but whose `finalize` methods have not yet been run. When control returns from the method call, the virtual machine has made a best effort to complete all outstanding finalizations.

The virtual machine performs the finalization process automatically as needed, in a separate thread, if the `runFinalization` method is not invoked explicitly.

The method `System.runFinalization()` is the conventional and convenient means of invoking this method.

**See Also:**

`Object.finalize()`

### traceInstructions

```
public void traceInstructions(boolean on)
```

Enables/Disables tracing of instructions. If the `boolean` argument is `true`, this method suggests that the Java virtual machine emit debugging information for each instruction in the virtual machine as it is executed. The format of this information, and the file or other output stream to which it is emitted, depends on the host environment. The virtual machine may ignore this request if it does not support this feature. The destination of the trace output is system dependent.

If the `boolean` argument is `false`, this method causes the virtual machine to stop performing the detailed instruction trace it is performing.

**Parameters:**

`on - true to enable instruction tracing; false to disable this feature.`

### traceMethodCalls

```
public void traceMethodCalls(boolean on)
```

Enables/Disables tracing of method calls. If the `boolean` argument is `true`, this method suggests that the Java virtual machine emit debugging information for each method in the virtual machine as it is called. The format

of this information, and the file or other output stream to which it is emitted, depends on the host environment. The virtual machine may ignore this request if it does not support this feature.

Calling this method with argument false suggests that the virtual machine cease emitting per-call debugging information.

**Parameters:**

`on - true to enable instruction tracing; false to disable this feature.`

### load

```
public void load(String filename)
```

Loads the native library specified by the filename argument. The filename argument must be an absolute path name. (for example `Runtime.getRuntime().load("/home/avh/lib/libX11.so");`). If the filename argument, when stripped of any platform-specific library prefix, path, and file extension, indicates a library whose name is, for example, L, and a native library called L is statically linked with the VM, then the JNI_OnLoad_L function exported by the library is invoked rather than attempting to load a dynamic library. A filename matching the argument does not have to exist in the file system. See the JNI Specification for more details. Otherwise, the filename argument is mapped to a native library image in an implementation-dependent manner.

First, if there is a security manager, its `checkLink` method is called with the `filename` as its argument. This may result in a security exception.

This is similar to the method `loadLibrary(String)`, but it accepts a general file name as an argument rather than just a library name, allowing any file of native code to be loaded.

The method `System.load(String)` is the conventional and convenient means of invoking this method.

**Parameters:**

`filename - the file to load.`

**Throws:**

`SecurityException - if a security manager exists and its checkLink method doesn't allow loading of the specified dynamic library`

`UnsatisfiedLinkError - if either the filename is not an absolute path name, the native library is not statically linked with the VM, or the library cannot be mapped to a native library image by the host system.`

`NullPointerException - if filename is null`

**See Also:**

`getRuntime()`, `SecurityException`, `SecurityManager.checkLink(java.lang.String)`

### loadLibrary

```
public void loadLibrary(String libname)
```

Loads the native library specified by the `libname` argument. The `libname` argument must not contain any platform specific prefix, file extension or path. If a native library called `libname` is statically linked with the VM, then the JNI_OnLoad_`libname` function exported by the library is invoked. See the JNI Specification for more details. Otherwise, the libname argument is loaded from a system library location and mapped to a native library image in an implementation- dependent manner.

First, if there is a security manager, its `checkLink` method is called with the `libname` as its argument. This may result in a security exception.

The method `System.loadLibrary(String)` is the conventional and convenient means of invoking this method. If native methods are to be used in the implementation of a class, a standard strategy is to put the native code in a library file (call it `LibFile`) and then to put a static initializer:

```
static { System.loadLibrary("LibFile"); }
```

within the class declaration. When the class is loaded and initialized, the necessary native code implementation for the native methods will then be loaded as well.

If this method is called more than once with the same library name, the second and subsequent calls are ignored.

**Parameters:**

`libname - the name of the library.`

**Throws:**

`SecurityException - if a security manager exists and its checkLink method doesn't allow loading of the specified dynamic library`

`UnsatisfiedLinkError - if either the libname argument contains a file path, the native library is not statically linked with the VM, or the library cannot be mapped to a native library image by the host system.`

`NullPointerException - if libname is null`

**See Also:**

`SecurityException`, `SecurityManager.checkLink(java.lang.String)`

### getLocalizedInputStream

```
@Deprecated
public InputStream getLocalizedInputStream(InputStream in)
```

**Deprecated.** *As of JDK 1.1, the preferred way to translate a byte stream in the local encoding into a character stream in Unicode is via the `InputStreamReader` and `BufferedReader` classes.*

Creates a localized version of an input stream. This method takes an `InputStream` and returns an `InputStream` equivalent to the argument in all respects except that it is localized: as characters in the local character set are read from the stream, they are automatically converted from the local character set to Unicode.

If the argument is already a localized stream, it may be returned as the result.

**Parameters:**

`in - InputStream to localize`

**Returns:**

`a localized input stream`

**See Also:**

`InputStream`, `BufferedReader.BufferedReader(java.io.Reader)`, `InputStreamReader.InputStreamReader(java.io.InputStream)`

### getLocalizedOutputStream

```
@Deprecated
public OutputStream getLocalizedOutputStream(OutputStream out)
```

**Deprecated.** *As of JDK 1.1, the preferred way to translate a Unicode character stream into a byte stream in the local encoding is via the `OutputStreamWriter`, `BufferedWriter`, and `PrintWriter` classes.*

Creates a localized version of an output stream. This method takes an `OutputStream` and returns an `OutputStream` equivalent to the argument in all respects except that it is localized: as Unicode characters are written to the stream, they are automatically converted to the local character set.

If the argument is already a localized stream, it may be returned as the result.

**Parameters:**

`out - OutputStream to localize`

**Returns:**

`a localized output stream`

**See Also:**

```
OutputStream, BufferedWriter.BufferedWriter(java.io.Writer),
OutputStreamWriter.OutputStreamWriter(java.io.OutputStream),
PrintWriter.PrintWriter(java.io.OutputStream)
```



Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms. Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.